

One LIMS Interface To Connect Them All

Exploring LIMS Agnosticity and Abstraction

By Siavash Mortazavi and Zohrab Broyan

Intro - The Hardwired Past

The legacy IAC application (ILASS predecessor) was hardwired to Illumina LIMS, making integration with other systems, such as Clarity LIMS, difficult. The root cause was a legacy architecture that did not account for flexible future integrations.

Because IAC was tightly coupled to Illumina LIMS, integrating with alternative LIMS solutions presented substantial architectural challenges.

ILASS Integration Design Goals

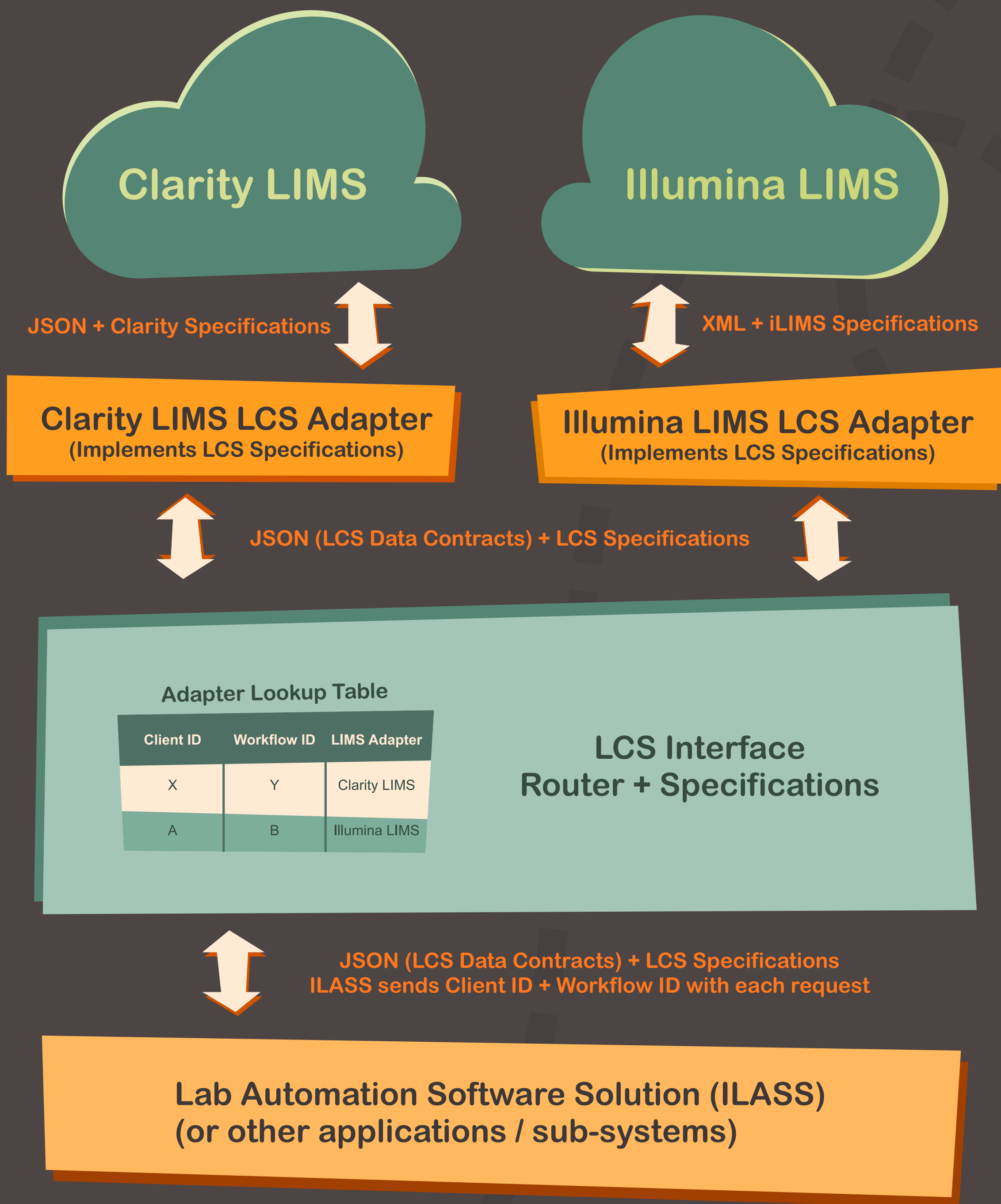
When developing the *Illumina Lab Automation Software Solution (ILASS)*, we recognized that a lab management platform must connect seamlessly to various external tools and services.

By defining the required interactions between ILASS and a LIMS as a set of public API specifications, any LIMS can implement an adapter to satisfy these requirements. We call this abstraction layer *ILASS LIMS Communication Services (LCS)*.

Business Advantage: A public LIMS connectivity specification allows clients to use their preferred (or existing) LIMS. This enhances ILASS's value and appeal, significantly.



Contact Us

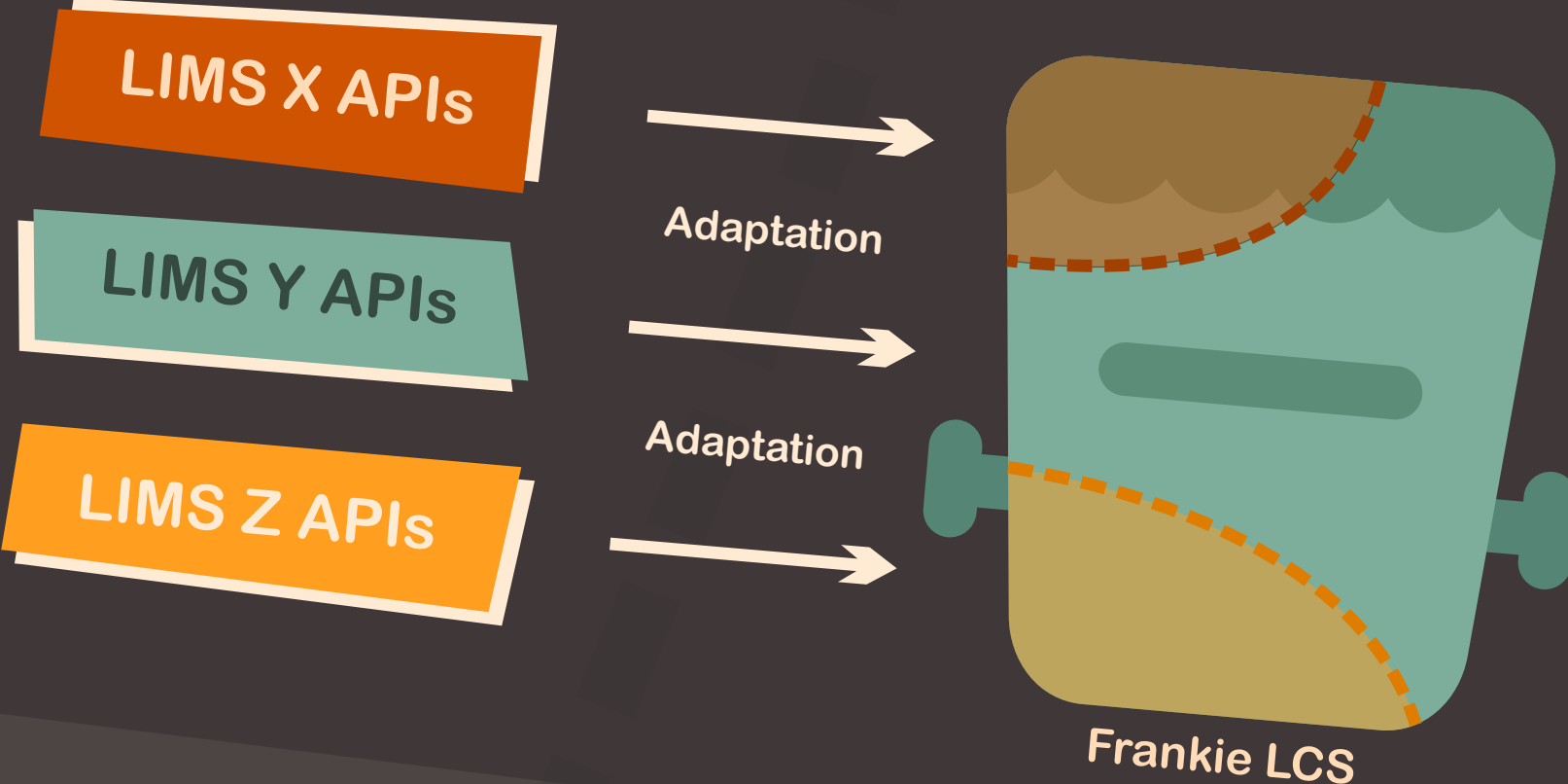


The initial LCS interface was implemented as a .NET class library (.dll) distributed to ILASS as a NuGet package. Its public interface enabled third-party developers to build LIMS adapters as .NET libraries. However, this architecture coupled third-party development tightly to the .NET ecosystem. To eliminate tech-stack constraints, we transitioned the LCS architecture to an HTTP-based Web API microservice. Internal and external teams can now build adapters against the LCS API specifications using the technology stack of their choice. Currently, we have adapters for Illumina LIMS, Clarity LIMS and Aquarium LIMS (Open-source).

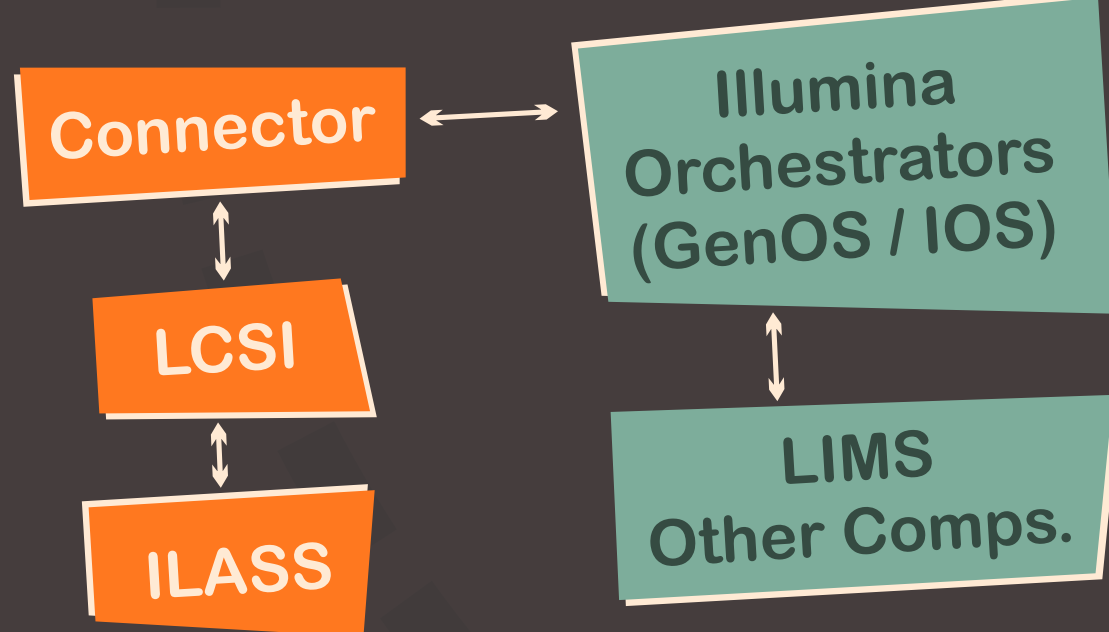
Implementation

Challenges

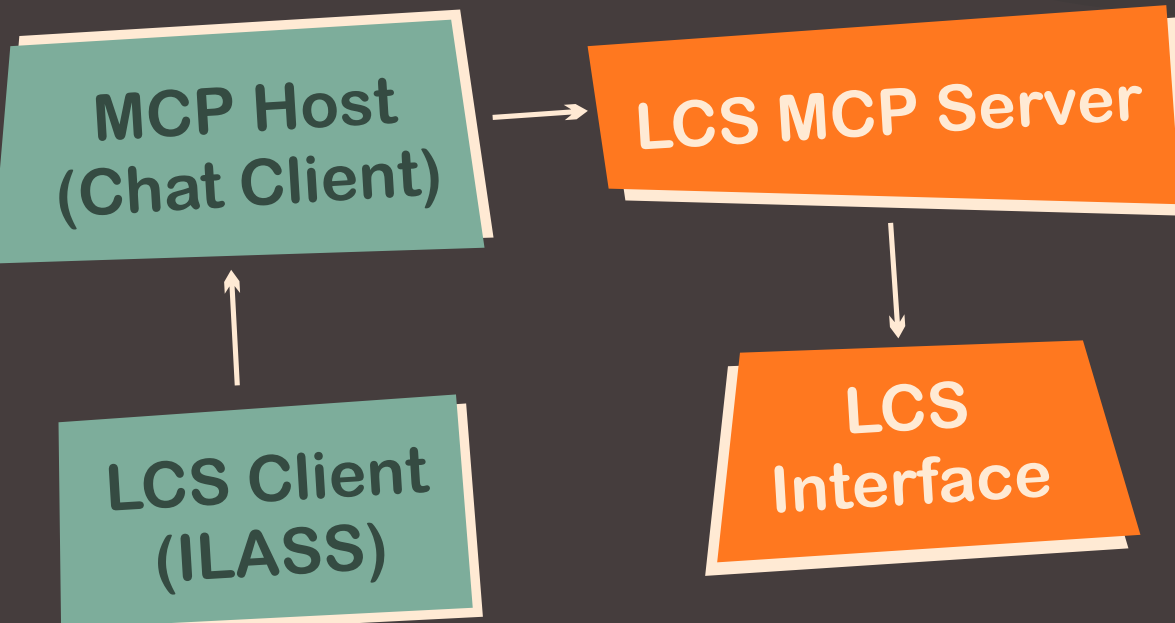
LIMS solutions often expose vastly different public API surfaces. Trying to adapt LIMSs with diverse paradigms risks turning the LCS interface into a "Frankenstein" APIs where only a subset of specifications applies to certain LIMS.



- More LIMS adapters (Gemini LIMS)
- More LCS client apps
- GenOS/IOS integration



- Standardize LIMS APIs across the industry
- LCS Model Context Protocol (MCP)



Future Possibilities